

Heritage

Matisse
kra
bts sio

Objectif	1
2. Analyse et Modélisation Initiale (Partie 1)	1
2.1. Création des classes de base	1
Mise en œuvre de l'Héritage (Partie 2)	2
3.1. Comparaison (Réponse Q8)	2
3.2. Réalisation technique	3
3.3. Redéfinition de méthode (Override)	3
Le Polymorphisme (Partie 3)	3
Abstraction et Généralisation (Partie 4 & 5)	4
5.1. Classe Abstraite	4
5.2. Diagramme de Classes Final (Réponse Q15)	4
Conclusion	5

Objectif

Développer une application console permettant à une banque de gérer ses clients et différents types de comptes bancaires (Courant et Épargne).

Ce projet avait pour but de mettre en pratique les piliers fondamentaux de la POO :

1. L'Encapsulation : Protection des données (attributs private, accesseurs getters/setters).
2. L'Héritage : Factorisation du code commun pour éviter la redondance.
3. Le Polymorphisme : Manipulation d'objets différents (Compte Courant, Épargne) via une interface commune.
4. L'Abstraction : Utilisation de classes abstraites pour définir des concepts génériques non instanciables.

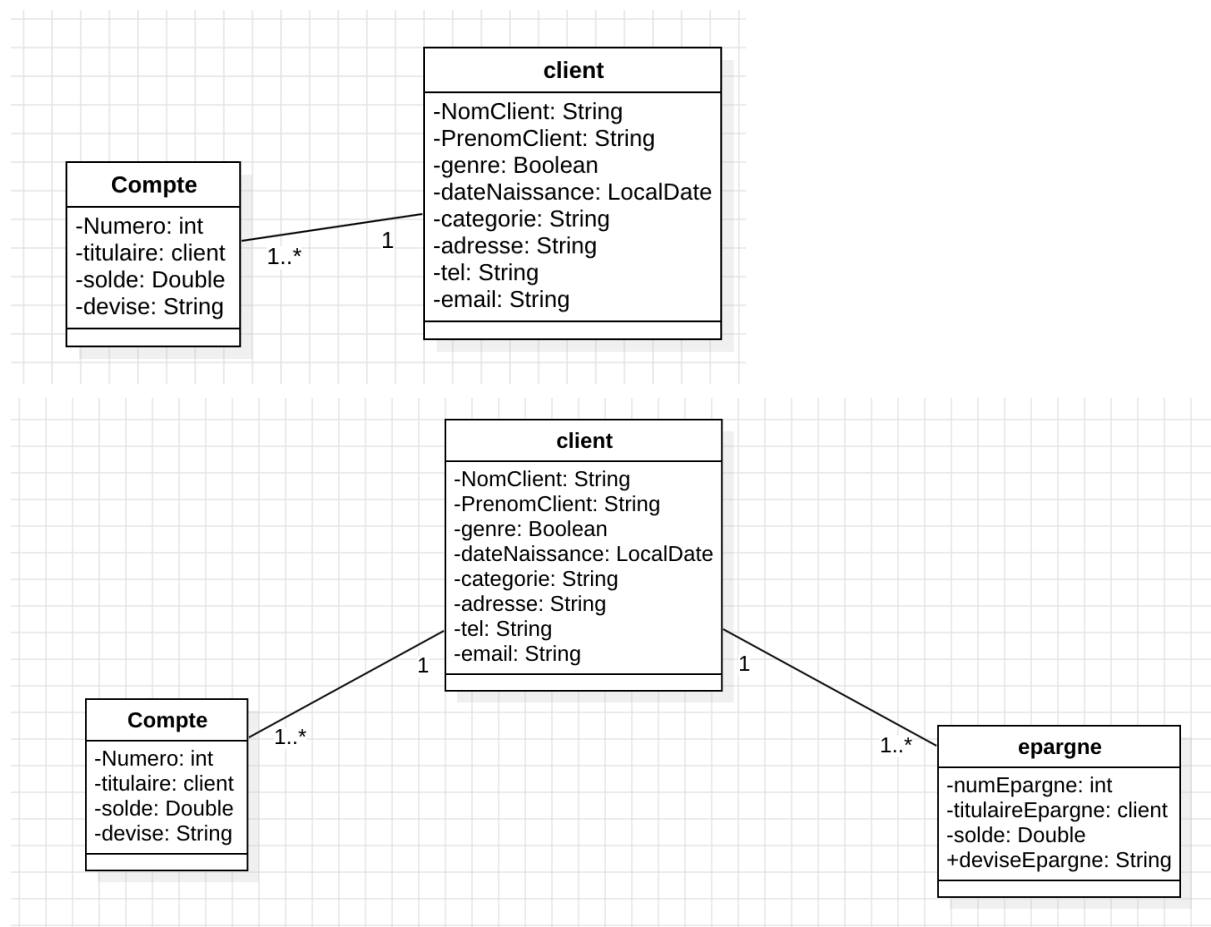
2. Analyse et Modélisation Initiale (Partie 1)

Dans un premier temps, nous avons modélisé les entités de base du système bancaire sans héritage.

2.1. Création des classes de base

Nous avons identifié deux classes principales :

- Classe Client : Contient les informations personnelles (nom, prénom, genre, etc.).
- Classe Compte : Représente un compte bancaire standard (numéro, solde, titulaire).



Choix techniques :

- Les attributs ont été déclarés en private pour respecter le principe d'encapsulation.
- Des accesseurs (getters) et mutateurs (setters) ont été générés pour interagir avec ces données.
- Une méthode decrire() a été implémentée pour retourner une chaîne de caractères résumant l'état de l'objet (ex: n° : 12345 - solde : 500 €).

```
//methodes
public void crediter(Double montant) {
    this.solde += montant;
    System.out.println("Crédit de " + montant + " effectué."); // Replace this use of System.out by a logger.
}

public abstract void debiter(Double montant);

public String decrire() { // On change void en String
    return "Compte n°" + this.numero + " - Titulaire: " + this.titulaire.getNomClient() + " " + this.titulaire.getPrenomClient() + " - Solde: " + this.solde + " " +
```

Mise en œuvre de l'Héritage (Partie 2)

Le besoin a évolué pour gérer un nouveau type de compte : le Compte Épargne. Plutôt que de réécrire tout le code, nous avons utilisé l'héritage.

3.1. Comparaison (Réponse Q8)

Avant de coder, nous avons analysé les différences entre un compte classique et un compte épargne :

Attributs Identiques (Hérités)	Attributs Spécifiques (Nouveaux)
Numéro, Titulaire, Solde, Devise	Taux d'intérêt

3.2. Réalisation technique

- Création de la classe CompteEpargne qui hérite de Compte (mot-clé extends).
- Utilisation du mot-clé super dans le constructeur pour appeler le constructeur de la classe mère et initialiser les attributs communs.
- Ajout de la méthode spécifique ajouterInteret() qui calcule les gains en fonction du taux et met à jour le solde.

```
package Model; // Rename this package name to match the regular expression '^([a-z]+)(\\.([a-z][a-z0-9_]*)*)$'.

public class Epargne extends Compte { // Hérite maintenant de CompteBancaire
    private Double tauxInteret;

    public Epargne(int numero, Client titulaire, Double solde, String devise, Double tauxInteret) {
        super(numero, titulaire, solde, devise);
        this.tauxInteret = tauxInteret;
    }

    // Nouvelle règle de débit imposée par le sujet
    @Override
    public void debiter(Double montant) {
        if (montant <= this.getSolde() / 2) {
            this.setSolde(this.getSolde() - montant);
            System.out.println(x: "Débit épargne effectué."); // Replace this use of System.out by a logger.
        } else {
            System.out.println(x: "Refusé : On ne peut pas retirer plus de la moitié du solde !"); // Replace this
        }
    }

    //methodes
    @Override
    public String decrire() {
        // Le sujet veut "taux = 5%" pas "Taux d'intérêt: 5.0%"
        return super.decrire() + " - taux " + this.tauxInteret + "%";
    }

    // ajouter_interet(): ajoute les intérêts au solde du compte (solde * taux d'intérêt / 100)
    public void ajouter_interet() { // Rename this method name to match the regular expression '^([a-z][a-zA-Z0-9_]*)$'.
        if(this.getSolde() > 0) { // on n'ajoute des intérêts que si le solde est positif
            Double interets = this.getSolde() * (this.tauxInteret / 100);
            this.crediter(interets); // on utilise la méthode de la classe mère pour ajouter les intérêts au solde
            System.out.println("Intérêts ajoutés : " + interets); // Replace this use of System.out by a logger.
        }
    }
}
```

3.3. Redéfinition de méthode (Override)

La méthode decrire() a été redéfinie dans CompteEpargne pour afficher le taux d'intérêt en plus des informations de base. C'est une première forme de polymorphisme.

```
@Override
public String decrire() {
    // Le sujet veut "taux = 5%" pas "Taux d'intérêt: 5.0%"
    return super.decrire() + " - taux " + this.tauxInteret + "%";
}
```

Le Polymorphisme (Partie 3)

Nous avons ensuite géré une liste de comptes hétérogènes (Comptes classiques et Comptes Épargne) appartenant à un même client.

Fonctionnement :

- Nous avons instancié une collection (ex: ArrayList<Compte>) contenant différents types de comptes.
- Lors du parcours de cette liste pour l'affichage, le programme détermine dynamiquement quelle version de la méthode decrire() utiliser (celle de Compte ou celle de CompteEpargne).
- Avantage : Cela permet de manipuler tous les comptes de manière uniforme sans se soucier de leur type précis.

```
List<Compte> tousLesComptes = new ArrayList<>();

tousLesComptes.add(new CompteCourant(numero: 111, listeDesClients.get(index: 0), solde: 2000.0, devise: "€", decouvertMax: 500.0));
// On ajoute un compte épargne pour le même client
tousLesComptes.add(new Epargne(numero: 123, listeDesClients.get(index: 0), solde: 5000.0, devise: "€", tauxInteret: 5.0));
```

```
for (Compte cpt : tousLesComptes) {  
    System.out.println(cpt.decrire())  
}
```

Abstraction et Généralisation (Partie 4 & 5)

Pour finaliser l'architecture, nous avons introduit des règles métier plus strictes :

1. Un compte bancaire générique ne doit pas exister (on ne peut pas créer juste un "compte", ce doit être soit Courant, soit Épargne).
2. Ajout du Compte Courant qui possède un découvert autorisé.

5.1. Classe Abstraite

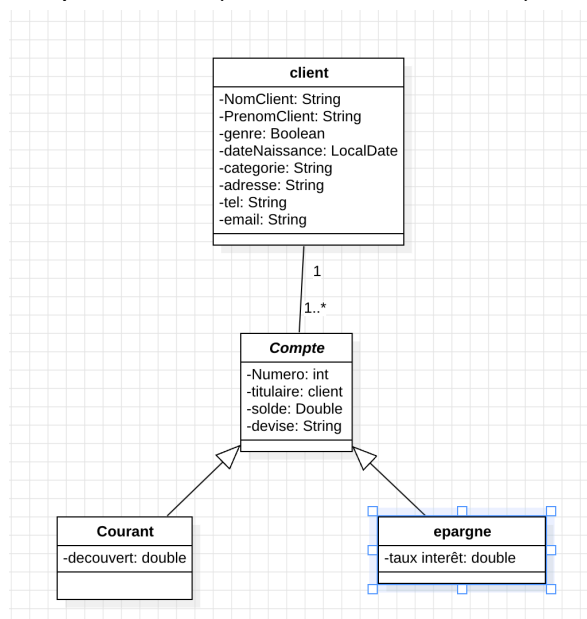
- Transformation de la classe mère en abstract class Compte.
- Elle factorise le code commun mais interdit l'instanciation directe (new Compte() est impossible).

```
public abstract class Compte {
```

5.2. Diagramme de Classes Final (Réponse Q15)

L'architecture finale se présente ainsi :

- Classe Mère (Abstraite) : CompteBancaire (attributs : numero, solde, titulaire).
- Classe Fille 1 : CompteEpargne (attribut : taux).
- Classe Fille 2 : CompteCourant (attribut : decouvertMax)



Matisse kra
bts sio

Conclusion

Ce TP a permis de valider la transition d'une programmation simple vers une architecture objet robuste. L'utilisation de l'héritage a permis de réduire la duplication de code, tandis que le polymorphisme et les classes abstraites ont rendu l'application évolutive (facilité d'ajouter un futur "Compte Jeune" ou "PEL" sans casser le code existant).